

Real-Time Object Detection on FPGA using Matched Filtering

Yaseen Alkhameri
Electrical and Computer
Engineering
University of California,
Davis
yalkhameri@ucdavis.edu

Ricardo Gonzales
Electrical and Computer
Engineering
University of California,
Davis
rigonzales@ucdavis.edu

Justin Hsu
Electrical and Computer
Engineering
University of California,
Davis
jchhsu@ucdavis.edu

Max Madrigal
Electrical and Computer
Engineering
University of California, Davis
momadrigal@ucdavis.edu

Isidro Pulido
Electrical and Computer
Engineering
University of California, Davis
isipulido@ucdavis.edu

ABSTRACT

Object recognition is a crucial AI/ML task that is conventionally implemented on general-purpose hardware like GPUs or CPUs. These devices are flexible but often have performance bottlenecks that limit their applicability to real-time settings in which ultra-low latency is critical. We propose a dedicated field-programmable gate array (FPGA)-based object detection system centered on matched filtering, replacing software execution with a hardware pipeline. By exploiting pipelining and spatial parallelism, the design reduces data movement and supports real-time template-based object detection from live camera input.

1. Introduction

Real-time object detection is a major area of computer vision used in embedded and autonomous systems such as self-driving cars, drones, robotics, and industrial inspection. In these applications, visual information must be processed continuously and quickly so that the system can respond to changes in its environment. Average performance alone is not enough. A detector may meet its target frame rate on average, but still fail if occasional delays cause missed or late detections. For safety-critical systems, the worst-case latency must be bounded and predictable. A delay of even a few frames can reduce the usefulness of the detection system or create unsafe behavior.

Cost, size, and power are also major constraints in embedded object detection. Small systems rely on physically small hardware to keep the footprint low, while cost-sensitive applications require cheap hardware solutions that software-based systems cannot provide. Power is especially important in battery-powered systems, where high energy usage limits operating time. Central Processing Unit (CPU) and Graphics Processing Unit (GPU) based systems are flexible, but they consume significant power because they support many general-purpose tasks. This extra flexibility can become unnecessary overhead when the system only needs to perform one fixed vision task.

Many modern object detection systems rely on complex software algorithms, including machine-learning-based approaches such as convolutional neural networks. Although these methods can achieve high accuracy, they often require significant computational resources, memory bandwidth, and power. Software must be run on power-hungry hardware resources such as CPUs or GPUs, but their prohibitive power consumption, high cost, and large physical footprint prevent their implementation in small and cheap systems. While CPUs and GPUs offer high generality and can perform a diverse range of computations, using them only for object detection is inefficient because their generality is not utilized. As a result, the extra functionality and hardware go to waste.

This motivates the use of domain-specific hardware for object detection. A domain-specific processor can reduce power consumption and physical footprint because the hardware only needs to process the computations required for object detection. There is very little hardware resource overhead because the hardware is designed for one specific task and does not need to account for unrelated workloads. This also reduces power consumption because no extra power is spent performing irrelevant operations. Application-specific integrated circuits (ASICs) can achieve even greater performance and efficiency, but they are much more costly and much more permanent in nature. Once a design is etched into silicon, it cannot be changed. For this reason, implementation on an FPGA was chosen. The high dynamic programmability of an FPGA allows designs to be changed easily, while its hardware-based implementation allows our design to function as a domain-specific processor. An FPGA is therefore a practical middle ground between general-purpose computation and hardware efficiency.

This project approaches object detection from a hardware-oriented perspective on an FPGA. The system is implemented on an Intel Cyclone V FPGA [1] using the DE1-SoC development board and is written in Verilog/SystemVerilog. It processes live video from the D8M camera and displays the detection output in real-time through a 640×480 Video Graphics Array (VGA) output. The design uses Sobel edge

detection as a front-end feature extraction stage before a matched-filter design [2] to detect fixed target objects in the camera stream.

First, the input image is downsampled and converted to grayscale. Next, a Gaussian filter is applied to filter out the noisy input. The Sobel operators extract edge information, which is passed into the matched filter, allowing the system to compare the shape of the target template against the edge structure in the camera frame. This allows the system to detect specific objects rather than only their presence. As a result, the system can both track the object and identify it.

The goal of this project is not to build the most complex detector possible, but to demonstrate how object detection can be applied to real-time systems using hardware for low-latency and high-throughput applications. Such systems can be used in autonomous vehicles, CCTV security systems, and other systems where the goal is to detect and track an object. Using an FPGA, our design can be economical, low power, high throughput, and retrofitted to adapt to any system. This project shows how a streaming detection pipeline can be implemented efficiently under hardware constraints. The design emphasizes simple control logic, reliable streaming behavior, and conservative FPGA resource usage. The following sections describe the hardware background, system architecture, and engineering tradeoffs that guide the final implementation.

2. Background

Before describing the system architecture, it is worth establishing what an FPGA actually is and what matched filtering does, since both are essential for our design project.

2.1 FPGA Architecture

An FPGA is built from a grid of programmable hardware blocks connected by a configurable network of wires. A CPU fetches instructions and runs them on a fixed datapath, but an FPGA lets the designer define the datapath itself. The resources we care about most are Lookup Tables (LUTs), Digital Signal Processing (DSP) blocks, and Block RAM (BRAM). LUTs are tiny memories that can be configured to implement any Boolean function and form the basic building block of custom logic. DSP blocks are hardened multiply-accumulate units optimized for arithmetic-heavy work, such as convolution and filtering. BRAM is a small on-chip memory distributed across the device for fast local storage. On the Cyclone V, BRAM resources are organized as M10K memory blocks, which are useful for line buffers and LUTs. The interconnect wires all of these blocks together according to whatever design has been loaded onto the chip. Each resource exists in a fixed quantity, so any design has to fit within the budget of the specific device it targets.

2.2 Convolution and Matched Filtering

Convolution is the core operation behind most image processing pipelines, including ours. A small grid of weights called a kernel is slid across an input image, and at each position, the overlapping pixels are multiplied by the corresponding weights and the products are summed to produce one output pixel. The choice of kernel determines the effect. A Gaussian kernel blurs the image, a Sobel kernel highlights edges, and a kernel shaped like an object of interest produces a strong response wherever that object appears in the frame. Cross-correlation is the same operation without flipping the kernel, and for the symmetric templates used in this project, the two are interchangeable. Matched filtering takes the kernel directly from a known image of the target object. When the filter is convolved with an input frame, the output peaks at locations where the frame locally resembles the template, and the position of that peak is where the object is. This is much simpler than a Convolutional Neural Network (CNN), but it maps naturally onto hardware because every position of the kernel runs the same multiply-accumulate structure, which an FPGA can build many copies of in parallel.

3. Design Goals

We first had to define a set of design goals in order to guide us on the project. Since the system processes live video directly on the FPGA hardware, we had to balance real-time performance, hardware limitations, and overall reliability. These goals helped influence the algorithms we chose, how the pipeline was implemented, and the tradeoffs made throughout the design process.

3.1 Low Latency

One of our main goals is to have the FPGA process and detect objects within the time constraints of live video. Every frame at VGA resolution must complete the full detection pipeline before the next frame arrives. Average frame rate alone is not enough. In safety-critical applications such as autonomous vehicles, a detector that hits its target throughput on average but occasionally stalls for several frames can cause real harm. A pedestrian missing for even 100 milliseconds at highway speeds translates to meters of uncovered distance. What matters is worst-case latency, the guarantee that every frame completes within a bounded time. CPUs and GPUs cannot provide this. Cache misses, OS interrupts, kernel scheduling, and contention from background processes introduce timing variance that is inherent to general-purpose software stacks. A dedicated FPGA pipeline avoids these sources of jitter. With no OS, no shared resources, and no scheduler, every pixel takes the same number of cycles through the pipeline on every frame.

3.2 Resource Efficiency

The DE1-SoC has a fixed budget of logic elements, DSP blocks, and on-chip BRAM, and our pipeline has to fit alongside the camera capture and VGA output logic that is already consuming part of the device. This made resource efficiency a constant pressure throughout the design rather than something we addressed at the end. A naive implementation that stores entire frames between stages would exceed available BRAM before we even got to convolution, so the pipeline streams pixels one at a time and only keeps the small neighborhood each stage actually needs. Choices such as sparse template matching and simplified Sobel preprocessing keep the design within the available hardware resource limits without sacrificing real-time throughput. These strategic modifications add up, and the final design utilizes well under a third of the device while still meeting timing constraints for live video.

3.3 Power Efficiency

Low power follows directly from low resource usage but also from the absence of overhead. A CPU or GPU running this same workload would spend a significant portion of its power budget on things unrelated to detection, such as cache hierarchies that move data between memory levels, branch predictors that guess the outcome of conditional code, and the OS itself keeping background services alive [3]. Our FPGA pipeline has none of that. Every transistor that switches does so because a pixel needs to be processed, and the parts of the device we are not using stay idle. The streaming structure helps here as well, since no large memory accesses are spinning up off-chip DRAM, just small line buffers and registers passing data between adjacent stages. The result is a design that runs the full pipeline at a Quartus-reported TDP of 524 mW.

4. Hardware Architecture

The FPGA architecture is built around a streaming pipeline. Instead of capturing an entire frame, storing it, and processing it all at once, pixels move through the system one at a time as they arrive from the camera. Each stage accepts one sample per cycle once its internal buffers are filled, so the design avoids full-frame storage while still processing live video. The final detection path consists of camera input, grayscale downsampling, Gaussian smoothing, Sobel edge extraction, sparse matched filtering, detection logic, and VGA output. The detailed top-level module connections are described later in the Implementation section.

4.1 Simplicity and Reliability

We implement our pipeline processing in a way that efficiently streamlines data through parallel functional units to meet the speeds required for real-time processing. Hardware bugs are expensive, and once synthesized, they can-

not be patched the way software can. So this pushed us towards using simpler signal processing algorithms (Gaussian smoothing, Sobel filter, matched filtering) compared to complex deep learning models on an FPGA. FPGAs not only allow the re-synthesis of hardware should any bugs arise, but since the type of computations between signal processing algorithms and deep learning algorithms are similar, hardware can be reused to implement more complex, deep learning-based models of detection. This provides the distinct advantage of implementing complex computations for a simpler objective, first enjoying easier optimizations for efficiency due to simplicity, before reusing that hardware for a much more complex system. That is to say, it is much easier to optimize hardware for a simpler system before transferring it to a complex system. And through the dynamic reconfiguration of the FPGA, hardware can be reused easily just by copying the code.

4.2 Efficient Resource Usage

The DE1-SoC FPGA has a finite amount of LUTs, DSP blocks, and on-chip BRAM, and our kernel is usually large (greater than 30×30) compared to typical CNN kernels. So fitting the full pipeline on-chip requires careful tradeoffs in how convolution is implemented. These tradeoffs extend beyond the convolution itself.

We also face design choices regarding how intermediate frame data is stored and which gradient magnitude best fits the approximation used in Sobel edge detection [4]. Frame data can be stored using massive line buffers to hold multiple rows of the input frame. This is inefficient because only a small local window is needed for each convolution operation. As such, the design uses streaming line buffers so only the neighborhood required for the current Sobel or matched-filter computation is stored in memory.

For edge extraction, we consider a more complex Canny-style processing, but the final hardware uses a simpler Sobel edge filter. This avoids additional buffering, complex angle calculation, and hysteresis thresholding. The Sobel output is sufficient because the matched filter only needs a consistent shape of the target object.

4.3 Streaming Video Pipeline

The camera provides horizontal sync (HSYNC) and vertical sync (VSYNC) signals to mark the start of each row and each frame, while the pixel clock flags when each pixel is valid. Each pixel flows through the stages one at a time, with intermediate results pipelined between modules. Pixels come in RGB values and are downsampled and converted into grayscale before Sobel edge extraction. The matched filter then scores the stream against the stored sparse template. The full streaming path is shown in Figure 1.

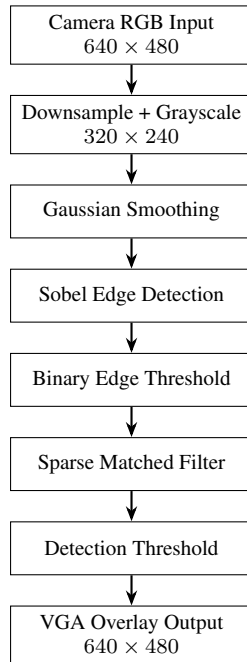


Figure 1: Final streaming video pipeline used for Sobel-based matched-filter detection.

4.4 Line Buffer Architecture

Convolution needs more than one pixel at a time. To compute the output for a single pixel using a 3×3 kernel, the hardware needs that pixel plus its eight neighbors, which means three pixels from the row above, three from the current row, and three from the row below. Since pixels arrive from the camera one at a time in grid order, the rows above and below are not available unless we store them somewhere first. Line buffers solve this. A line buffer is a small on-chip memory that holds one row of pixels, and by chaining a few of them together, the pipeline can keep the last few rows of the image on hand at all times. As each new pixel comes in, it gets pushed into the buffer chain, and the old pixel at the other end falls out. The hardware can then reach into the buffers and pull out the small neighborhood it needs for the current convolution. This same approach is reused at every stage that needs neighborhood data, including Gaussian smoothing, Sobel, and matched filtering, with each stage sized to match the footprint of its kernel.

4.5 Parallel Processing

FPGAs are widely used for their ease of implementing a massively parallel system. Parallel systems are a horizontal integration that heightens throughput through multiple instances of the same functional blocks. Each functional block can process data independently, so the system can effectively become many times faster depending on how many parallel functional blocks there are in the system. For example, in

convolution done through general matrix multiplication using the “im2col” function (a function that transcribes all the relevant data from the input matrix and kernel to a convolution into vector form so the convolution operation can be performed using a matrix multiplication) [5], parallel processing can drastically speed up computation. Through parallel functional blocks, multiple rows of the input matrix can be run through the convolution calculation at once, or multiple filters can be applied to one input at once, or a combination of multiple inputs and multiple filters can be processed at once. The more parallel functional blocks, the more independent calculations can be done at once, thus the higher the throughput of the system.

4.6 Why FPGAs Are Well Suited for Parallelism

Parallelism can be implemented efficiently with FPGAs because the hardware is dynamically routed to fit the current system design [6]. Parallel functional blocks can be instantiated infinitely as long as there is available hardware, making FPGAs highly customizable and amenable to parallelization. This property of dynamically instantiating hardware is unique to FPGAs, as other systems, such as CPUs, GPUs, ASICs, Tensor Processing Units (TPUs), etc., all have hardware that is already set in stone. The user can choose to utilize fewer hardware resources (for low power considerations, for example), but cannot choose to use higher levels of parallelization because the hardware does not exist to support it. This results in FPGAs acting as a “sweet spot” for hardware, as their dynamic natures not only allow for design prototyping, but their support for highly parallel hardware makes them a stand-out option for machine learning applications such as our project.

4.7 FPGAs vs. GPUs

The ability of an FPGA to implement domain-specific hardware drastically improves its utilization when compared with GPUs. While GPUs are also highly parallel hardware, they are built to support a diverse range of functions and operations. As a result, they are not highly optimized for performing one specific task but rather performing a range of tasks, making them inefficient when compared with FPGAs. FPGAs can implement hardware specifically for performing one task, thus maximizing hardware utilization because they only need to support that one task. Furthermore, FPGAs are dynamically reconfigurable—they can instantiate as much parallel hardware as they physically support. This provides further optimizations as GPUs are permanently etched into silicon. They cannot choose to use fewer hardware resources because those resources are already there and consuming power. They cannot choose to use more hardware resources because only so many functional units are physically etched onto the silicon. FPGAs can support choosing to use more or fewer hardware resources by simply instantiating more or fewer hardware blocks. This means FPGAs can not only be

more power efficient than GPUs in terms of specificity of hardware functionality and hardware utilization, but FPGAs can also provide higher throughput than GPUs because of the domain-specific hardware not needing to support any other task [7].

4.8 Parallelism in Our Design

Our project uses parallelism at the level that best matches the final hardware. The RGB-to-grayscale stage forms the weighted red, green, and blue terms in parallel, so a grayscale sample can be produced each cycle. The Sobel stage computes the horizontal and vertical gradients with combinational add/subtract logic over a 3x3 window, then thresholds the L1 magnitude into a binary edge stream. The matched filter is also parallel, but it is not implemented as the final systolic-array datapath. Instead, the final matcher gathers the 64 nonzero sparse template taps in parallel and reduces their signed products through a pipelined score tree. This keeps throughput at one score per cycle after fill while avoiding the hardware cost of computing every dense template position.

4.9 Memory Considerations

Memory usage is a significant concern in this system due to the high memory cost of a standard definition video feed and limited FPGA memory resources. As a result, the original 640×480 resolution is 2:1 downsampled to 320×240 . Line buffers are used to store the first 3 rows of the downsampled input video data. Convolution is then performed on those 3 rows, then once the filter finishes across the entire row, the first row is discarded, and then the fourth row is pulled into the line buffer, so convolution can be performed on the second row, streaming the relevant data and reusing hardware. Tiling [8] is also implemented on the input data. To save memory, only a specific region of the input frame data is pulled into the line buffer across the row. This way, instead of having a large memory structure necessary to store entire rows of frame data at once, only a portion of the frame needs to be stored-the portion necessary for the current convolution computation. This tiling allows for much smaller memory structures. Smaller memory structures mean either less BRAM is used, saving capacity to be used elsewhere, or smaller registers are instantiated.

4.10 Matched Filtering

The core of the detection architecture relies on matched filtering implemented directly on FPGA hardware. To support this system, a Python script was created to utilize the Open Source Computer Vision (OpenCV) library [9] to generate the weights for detecting that specific object. OpenCV looks across the entire image and captures defining characteristics in the weight. This can be any defining characteristic, such as contrast gradients (edges and boundaries), volumetric shading (where light and dark spots are on the object), texture, etc. OpenCV then extracts these features in a

weight matrix, and this matrix is then used as the kernel for our object detection convolution calculation. While this convolution calculation is incredibly effective at finding exact matches, it relies on the input's spatial invariance, so the object must be at the same scale and orientation as the object from the template; otherwise, the features will fail to match. Firstly, if the input contains features of the object of interest, but this object is a different size than the object used to develop the weight filter, then the confidence of object detection is significantly reduced-that is to say, the system is uncertain in its prediction and is randomly guessing instead of coming to an evidence-backed conclusion. A smaller or larger object in the input means a change in the overall resolution of the object, resulting in certain features not being in the same area where the weights representing them are. In other words, this drastically skews the convolution calculation, as even though visually those features are there, they have shifted in position and therefore do not line up with the filter. This means that when convolution is computed, the shifted input values do not result in an expected magnitude output when compared with the original, thus throwing off the calculations and drastically impairing the system's ability to detect an object. Without a matching expected magnitude output to the convolution, the system cannot determine whether the object is present because, aside from the definitions and spatial location of the edges as defined by the expected output, the system does not know what makes an object an object. Secondly, using a fixed template poses challenges in detection when the object of interest is rotated. Similar to the aforementioned size mismatch, a rotated object shifts where the pixels of interest used to develop the weights are. Once again, this produces an unexpected magnitude output, causing failure in object recognition for the same reasons as the scale mismatch. The convolution output does not match the expected output, and since the system is incapable of defining what an object is beyond its spatial characteristics, the system is unable to detect that object. To capture rotated objects, the following calculation must be applied:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

As we see, further matrix multiplication must be performed, as well as finding the $\sin \theta$ and $\cos \theta$ of the angle. While matrix multiplication is still a complex operation requiring many multiply-accumulate operations and additional hardware, the real problem is calculating the trigonometric functions [10]. $\sin \theta$ and $\cos \theta$ are non-linear functions that must either be approximated with a power series (incredibly expensive due to the amount of multipliers, dividers, and adders required to implement):

$$\sin x = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{(2n+1)!} = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$\cos x = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n}}{(2n)!} = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

or using lookup tables to store and search for values (requiring a massive amount of hardware and memory resources), or utilizing the Coordinate Rotation Digital Computer (CORDIC) algorithm to calculate the values. No matter which method is chosen to find these values, it is both an additional step within the pipeline, increasing latency, and requires additional hardware, potentially using resources that could be used elsewhere. Even though humans are visually able to deduce that it is the same object, merely shifted in some manner, object detection systems such as these are not able to. This is due to the significant hardware constraints necessary to implement this [11]. One way of accounting for such changes in the image is to keep a library of all possible rotated orientations, along with all possible sizes. This could potentially result in hundreds or thousands of filters, and all these filters must be applied simultaneously to the input to detect such a change. Additionally, all of these parallel filters must be calculated in real-time for our system. The hardware necessary to perform such a task far exceeds what is capable with our FPGA, and thus is not a feasible solution. A second way to account for such changes is through a deep neural network. Through the many layers of the network, the model can extract such shifted features, but the hardware necessary to implement inference for such a model is also no easy feat. The required hardware is still greater than what the FPGA offers, and thus is not possible on our current system. As such, our system is subject to strict guidelines when performing object detection-the object of interest must possess the same invariant spatial properties as the object used to develop the filter (i.e., the same scaling, rotation, translation, etc.), otherwise the processing required to account for such shifts would become prohibitively expensive and not feasible for our FPGA. As such, our system is incapable of accounting for such changes and thus will fail should any of these changes occur.

4.11 Template Representation

Template size is directly correlated to the resolution of the features of the object of interest. The larger the template, the more features can be extracted from the object of interest, thus a more detailed match can be performed, and the higher the confidence level of matched detection. Due to the increased size, there is more available space in the kernel to represent finer details. However, template size comes at great hardware cost. The number of computations required for use of a larger filter is quadratic in nature because as the kernel size increases by 1, the amount of computations increases

with the square of the kernel dimensions, since the filter area grows as k^2 . This type of scaling leads to significant challenges in using large kernels. Not only must the large kernel and all relevant operations for that kernel now be performed, but the hardware to handle that computation is now more complex because the registers, multipliers, and adders must be wider to support the increase in size. Furthermore, the line buffers, the amount of parallel hardware to support real-time computation for video, and the dramatic increase in routing complexity to connect all those hardware resources also increase. As a result, a balance must be struck between finding the optimal size kernel to capture all necessary information to determine object recognition and hardware complexity.

The sparse template is generated offline and stored as a set of tap locations and signed weights. The script is completely parameterizable, allowing different template sizes and sparse tap numbers. Figure 2 shows an example template overlay of a 64x64 cow template with 128 taps. Green taps represent positive weights that reward matching object edges, and red taps represent negative weights that penalize background or non-object regions.

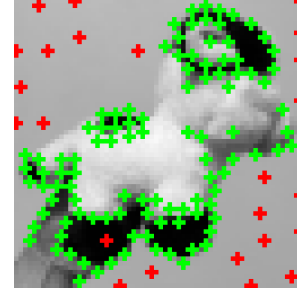


Figure 2: Sparse template tap overlay.

4.12 Sobel Edge Detection

Noise is of critical importance to mitigate, as anything other than the object of interest can be categorized as noise. As such, a Gaussian blur is applied. A Gaussian blur acts not only as a low-pass filter, filtering out all the high-frequency noise, but also suppresses noise through averaging a pixel's value with its neighbors based on their distance from the center. This helps Sobel edge detection, as the random fluctuations in pixel intensity (that will be interpreted as edges) are instead filtered out and ignored. Sobel edge detection is implemented using the Sobel gradients: The Sobel operator uses two convolution kernels to approximate the horizontal and vertical image gradients.

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad (1)$$

$$G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad (2)$$

Through convolution, these two filters are applied across the entire input frame, where the largest changes in pixel intensity can be found. The areas with the highest gradients count as edges and remain in the image. The remaining pixels without significant gradients are discarded, leaving behind a contrasting image where edges are highlighted and non-edges are suppressed.

The gradient magnitude is then calculated using:

$$|G| = \sqrt{G_x^2 + G_y^2} \quad (3)$$

In hardware, the exact square-root magnitude is expensive because it requires a square-root unit, an iterative approximation, or a LUT. The final design uses a simplified Sobel magnitude approximation before thresholding. Pixels with enough edge strength are kept as edge pixels, while pixels below the threshold are suppressed. This produces the edge representation used by the matched filter.

5. Design Decisions and Tradeoffs

Throughout the design process, several architectural choices were evaluated to determine the most practical implementation for a real-time FPGA-based object detection system. In many cases, improving one aspect of the design came at the expense of another, requiring careful consideration of performance, accuracy, hardware complexity, and resource consumption. The final system reflects a series of engineering tradeoffs intended to meet the project’s goals of low-latency, reliability, and efficient use of FPGA resources. The following section discusses the key design decisions that shaped the final architecture.

5.1 Gradient Magnitude Approximations

During early filter testing, the project evaluates Sobel edge detection as a possible feature extraction stage for the object detection pipeline. The Sobel operator computes horizontal and vertical image gradients, referred to as G_x and G_y . These values estimate how quickly image intensity changes in the horizontal and vertical directions. A large gradient magnitude usually corresponds to an edge, which is useful when detecting object shape while reducing dependence on raw pixel intensity.

The mathematically exact gradient magnitude is shown in Eq. 3. This expression gives the Euclidean magnitude of the gradient vector and is the most accurate representation of edge strength. However, it is expensive to implement directly in FPGA hardware. Squaring both gradient components requires multipliers, adding the squared values increases bit width, and computing the square root requires either a dedicated square-root unit, an iterative algorithm, or a lookup-table approximation.

A less expensive alternative is the squared magnitude:

$$|G|^2 = G_x^2 + G_y^2 \quad (4)$$

This removes the square-root operation while preserving the relative ordering of gradient strengths. However, this method still requires multiplication for both G_x and G_y , which can consume DSP blocks or additional logic resources.

The project also considers simplified norm approximations that avoid multipliers and square-root hardware. The L1 approximation

$$|G| \approx |G_x| + |G_y| \quad (5)$$

only requires absolute-value logic and an adder, making it easier to pipeline in a real-time video stream. The maximum approximation

$$|G| \approx \max(|G_x|, |G_y|) \quad (6)$$

is even cheaper because it only requires absolute-value logic and a comparator, but it can underestimate diagonal edges because it ignores the smaller gradient component.

For this design, the L1 approximation provides a practical balance between hardware simplicity and edge quality. It avoids multipliers, dividers, and square-root units while still producing large output values for strong edges. This supports the main system goals of low-latency, reliable timing, and efficient FPGA resource usage.

5.2 Canny vs. Sobel Edge Detection

During development, we tested a Canny-style edge pipeline before the matched filter [12]. This version used Sobel gradients, a Cartesian-coordinate direction estimate, 45-degree direction approximation, non-maximum suppression (NMS), and hysteresis thresholding. The direction logic avoided a full `atan` calculation by binning the gradient direction from G_x and G_y , which allows the NMS stage to compare each pixel against neighbors along the edge direction.

The Canny pipeline produces thinner and cleaner edges, but it adds memory and DSP cost. In the prototype fit report, the Canny core uses 10 M10K blocks and 14 DSP blocks [13]. These resources come from the direction suppression and hysteresis stages. In the final design, the `sobel13x3_stream` module uses 1 M10K block and 0 DSP blocks [14]. Although the final Sobel module uses more ALMs due to later pipelining, it avoids the heavy memory and DSP-heavy stages of the Canny prototype. Since the matched filter only needs a stable shape, the final system uses Sobel edge extraction before matched filtering. This keeps the pipeline easier to tune, lowers memory and DSP usage, and still preserves enough object information for detection.

5.3 FFT Convolution vs. Spatial Convolution

Convolution and matched filtering can be implemented either in the frequency domain or directly in the spatial domain. FFT convolution is attractive for very large dense kernels because convolution becomes point-wise multiplication after the input and kernel are transformed into the frequency domain [15]. For this project, that overhead is not a good fit.

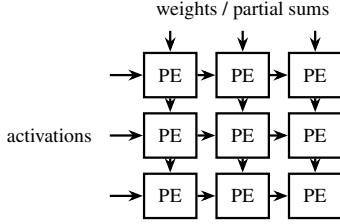


Figure 3: Conceptual systolic-array matcher structure.

The pipeline works on a live pixel stream, and the template is sparse rather than dense. Transforming image regions into and out of the frequency domain would add buffering, control complexity, and latency before the design even reaches the 64 useful template taps.

The final system uses spatial-domain filtering. Sobel and Gaussian filtering operate on small 3x3 neighborhoods, which are naturally supported by streaming line buffers. The object matcher also stays in the spatial domain, but it does not compute a dense convolution over every template location. Because the stored template contains only 64 nonzero signed taps, the final hardware gathers those positions directly and reduces them through a score tree. This keeps the computation aligned with the actual sparsity of the template.

5.4 Score Tree vs. Systolic Array

A systolic array was developed and tested as an earlier matcher architecture. In that approach, a two-dimensional grid of processing elements performs multiply-accumulate operations while activations and partial sums move across the array. Figure 3 shows the idea at a high level. This structure is useful for dense sliding-window computation because many nearby output positions can reuse the same incoming data, and a weight-stationary arrangement can reduce repeated weight reads [16]. The cost is that the array needs enough processing elements, alignment delays, and control logic to keep the grid fed correctly.

The final design uses the score-tree structure in Figure 4 instead. The template is stored as a list of `{row, col, weight}` entries, so `score_tree.v` only gathers the 64 selected edge samples from the template window. Each selected sample is multiplied by its signed template weight, and `score_mac_tree.v` reduces the 64 products through a pipelined adder tree. For 64 taps, the MAC tree has six reduction stages, and the sparse tap gather adds one registered stage before reduction.

This score-tree datapath matches the final hardware better than the systolic array because the template is sparse. A dense 64×64 template would contain 4096 positions, but the active design uses only 64 signed taps. A full systolic array would still need to move, align, or ignore many zero-weight positions. The score tree does the opposite: it starts from the sparse representation and instantiates arithmetic only for the

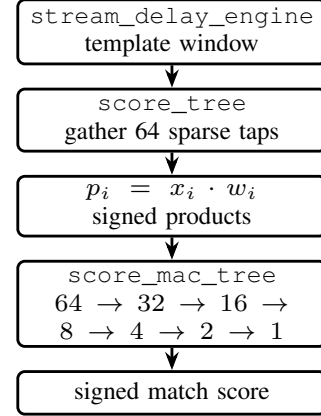


Figure 4: Score-tree structure used by the final sparse matched filter. The datapath gathers only nonzero template locations and reduces their signed products through a pipelined adder tree.

nonzero taps. The tradeoff is that the score tree is specialized for sparse fixed-template matching, while the systolic array is more general and remains useful for dense convolution or future learned-filter designs.

5.5 Sobel Edge Detection vs. Grayscale Matched Filtering

A matched filter can be applied to either the raw grayscale image or the output of an edge detector. Both work, but the choice changes how much the filter actually sees. On raw grayscale, the filter matches against pixel intensities. This means lighting becomes part of the template, which matters because differences in lighting directly change the gradients the filter is trying to match. The same object under a brighter lamp or against a different background looks different to the filter, even though to a human it is clearly the same thing. A template trained on one lighting condition will fail in another, which is a serious limitation for any system pointed at a real scene with changing light.

Running the filter on the Sobel output sidesteps most of this. Sobel responds to changes in intensity rather than the intensity itself, so a dark object on a light background and a light object on a dark background both produce strong edges in roughly the same places. The template is matching against the shape of the object rather than its appearance, which is more robust to lighting and to small changes in color. There is a tradeoff, since edges throw away most of the information in the image, and small features can get smoothed out by the Gaussian filter that runs before Sobel. But for our objects, which are distinct and have well-defined outlines, the edge representation captures more than enough to identify them and does so consistently across lighting conditions. This is why our pipeline runs the matched filter on the thresholded Sobel output rather than on grayscale pixels directly.

5.6 Fixed Template vs. Learned Model

Fixed templates rely on computer vision systems like OpenCV to generate the template [17]. While OpenCV is very powerful, it relies on pre-designed algorithms to extract certain features. As a result, it is limited in the range of images. If anything complex is introduced that was not accounted for when designing OpenCV, the library can struggle to extract features. Furthermore, OpenCV struggles with "real-world" complexity, such as rotated objects and occlusion. These features are hard to define using algorithms, and as a result, hinder the effectiveness of OpenCV. However, OpenCV is very computationally efficient due to this predictable nature. It can be run easily and does not require large amounts of training data. This presents OpenCV as an excellent option for most trivial image recognition tasks, as, through pattern recognition and matching, a very accurate result can be obtained.

Learned models require machine learning models like convolutional neural nets to operate [18]. Large amounts of training data are required for the model to learn and extract features from. Furthermore, training and inference of CNNs are more computationally expensive compared to OpenCV. Despite this computational complexity, CNNs can be very effective in image recognition. They can extract extremely complex features, thereby being able to perform object detection with incredible accuracy. CNNs are extremely performant for complex frames, being able to find exactly what it wants due to the complex feature extraction.

For this system, fixed templates were used. The objects we sought to perform object detection on were all distinct, simple objects, like a banana or toy car. While rotations of the object or overlapping the object with a noisy background cause interference, the modules for matched filtering and Sobel edge detection are the solution.

5.7 Pipelining vs. Latency

Our design is fully pipelined; registers are used to capture intermediate signals between each module. This allows us to run the system at a high enough operating frequency to support real-time processing necessary for object tracking and edge detection. Since each piece of data of interest is stored between modules in registers, the clock signal can safely transition without losing data due to slack or hold time violations. This means the addition of registers to capture intermediate data effectively shortens the datapath and allows for faster clock signal transitions. These registers provide sufficient time before the clock edge arrives for the data to be fully processed, giving us ample setup slack, while also maintaining sufficient time after the clock edge arrives for the upstream data to finish processing before sending out the latched data, giving us positive hold slack. That is to say, we can shorten the datapath using registers, allowing for faster operating frequency, while simultaneously ensuring there is

sufficient processing time for upstream data so it would not be corrupted by the currently latched data "catching up to" the upstream data and causing a collision.

Of course, pipelining does come at a cost. The extra registers in between modules take up more area and physical hardware, and the control logic and finite state machine to support pipelining have to account for the capture and release of that data. Furthermore, since pipelining requires data to be captured and held while the clock signal transitions, there is inherently more latency introduced by pipelining as more clock cycles are needed to complete a task. However, this drawback is mitigated by the increase in clock operating frequency that pipelining allows for. Furthermore, relying on a one-cycle system would require such a low clock operating frequency that it would not be possible to perform the computations necessary for real-time object tracking and edge detection. This hypothetical one-cycle system would have such a long datapath between the first and last modules that the clock would have to run at such a slow operating frequency just to account for this path. As a result, live processing for smooth, 60 FPS video input would not be possible.

5.8 Large Template vs. Small Template

Larger kernels obviously provide more resolution and information in the object of interest. Because there are more weights, more complex features can be found using convolution, thus the higher the confidence of object detection. However, larger kernels also mean significantly more complex hardware. Larger registers and memory structures are needed to store the kernels, and wider adders and multipliers are required to perform computation. This drastically increases the amount of area needed to implement such a design, and takes up more memory and hardware resources in a resource-limited system.

Small templates, while not being able to convey as much detail as larger templates, are significantly easier to work with. In addition to using simpler hardware compared to the larger templates, the amount of calculations is much less when using smaller templates. In a system where processing live, 60 FPS video is the goal, this shortened latency is of utmost importance.

For this system, object detection is only performed on distinct objects-their shapes are so different that small details are not needed to differentiate them. As a result, small templates can be used due to this uniqueness, saving a great amount of both hardware and memory resources.

6. Implementation

This section walks through how the design was built and brought up on hardware. It covers the FPGA board used, how live video gets in and out of the device, how the streaming pipeline stays synchronized across stages, and how the individual Verilog modules connect to form the full system.

6.1 FPGA Platform

As mentioned earlier, the detector runs on the Terasic DE1-SoC, built around an Intel Cyclone V SoC (5CSEMA5F31C6). What matters for this section is what the board gives us to work with: a Cyclone V FPGA with about 32,000 ALMs (Adaptive Logic Modules), 4,450 Kbits of M10K memory, and 87 DSP blocks, a 50 MHz onboard oscillator that feeds our internal PLLs, a VGA output port, and a General-Purpose Input/Output (GPIO) header that mates with the D8M camera module. The D8M provides the input stream, carrying an OmniVision OV8865 sensor and a MIPI-to-parallel bridge that hands pixels to the FPGA along with the HSYNC, VSYNC, and pixel clock signals from section 4.3. We run the camera at 640×480 . Figure 5 shows how these pieces fit together.

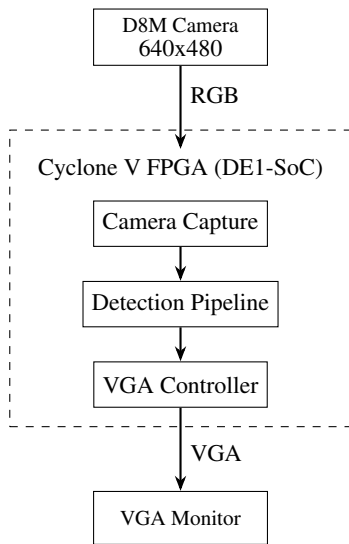


Figure 5: FPGA Hardware platform.

6.2 Video Interface

The detector sits between two video interfaces: the D8M on the input side and a VGA monitor on the output side. Both need their own handling because neither matches the format or clock rate the pipeline runs on internally.

On the input side, the OV8865 sensor inside the D8M sends 10-bit Bayer RAW pixels rather than RGB. A MIPI bridge brings the serial sensor data onto the FPGA pins, and a RAW-to-RGB stage interpolates the missing color channels at every pixel. From there, the stream is downsampled 2:1 in each dimension from 640×480 to 320×240 . This reduces the pixel processing rate from 307,200 to 76,800 pixels per frame, a 75% reduction in frame pixel workload. The stream is converted to 8-bit grayscale using an ITU-R BT.601 weighting, which is the resolution and format that the rest of the pipeline runs on. Downsampling by a factor of four in pixel count keeps the pipeline within budget and still leaves enough detail to identify the objects we care about.

On the output side, an on-chip PLL produces a 25 MHz pixel clock for the standard 640×480 at 60 Hz VGA mode. A VGA controller drives HSYNC, VSYNC, and the color outputs from that clock and a pair of horizontal and vertical counters. Just before the controller is a debug mux that selects which signal to display: the raw camera RGB, the Sobel edge magnitude, a signed matched-filter score heatmap, or the threshold mask. A separate switch overlays the detection bounding box on top of whichever view is active. We used this constantly during bring-up to check each stage, and it is useful for demos since the same bitstream can show every intermediate output.

6.3 Pipeline Timing

Every stage in the pipeline does some work before producing its output, so a pixel that enters the system at cycle 0 does not come out the other end at cycle 0. The 3×3 front-end filters need to wait until their local windows are available before producing valid samples, and the matched filter needs the rows that contain the template taps. Counting the registered Gaussian and Sobel front-end stages, the template-window delay, the one-cycle sparse tap gather, and the six-stage MAC reduction tree, the processing path to a valid match score is about 72 cycles, or about 2.88 microseconds at a 25 MHz pixel clock. This number describes the fixed registered datapath latency after the stream is active; the first valid output at the start of a frame also depends on the line buffers filling their spatial windows. Table 1 shows the fixed pipeline delay. Once the pipeline is filled, every stage produces one new output per cycle.

Two things are critical for keeping the pipeline correct under this latency. First, every stage outputs a valid signal alongside its data. When a stage is still filling up its line buffers or has not yet built a complete window, it drops the valid signal so downstream stages know to ignore whatever happens to be on the data lines. Once the stage starts producing real outputs, valid goes high and stays high for the rest of the frame. Downstream stages only update their state when they see valid input. Second, every stage carries the pixel coordinates (x, y) alongside the data and valid signals. This is what lets the detection logic at the end of the pipeline know which pixel in the original image a given score corresponds to, even though that pixel left the camera dozens of cycles earlier. Without these coordinates traveling with the data, there would be no way to map a peak in the matched filter back to a position on the screen.

Table 1: Fixed pipeline latency for one valid matched-filter score.

Pipeline stage	Cycles	Running total
Gaussian + Sobel stages	2	2
Template-window delay	63	65
Sparse tap gather	1	66
MAC reduction tree	6	72
Total latency	72	72

The final matched-filter score has a fixed latency of 72 clock cycles. This comes from the registered Gaussian and Sobel stages, the template-window delay, the sparse tap gather stage, and the pipelined MAC reduction tree:

$$2 + 63 + 1 + 6 = 72 \text{ cycles.}$$

6.4 Module Integration

The full design is tied together at the top-level module `DE1_SOC_D8M_LB_RTL.v`, which instantiates each pipeline stage, the camera infrastructure, the VGA controller, and the user-facing support modules, and routes signals between them. Every module that touches the pixel stream uses the same interface convention: a clock, an active-low reset, a data signal, a single-bit valid, and a pair of `x_in/y_in` coordinates that travel alongside the data. Outputs follow the same pattern with `x_out` and `y_out` updated to whatever position the new sample corresponds to. This convention is what made the integration straightforward: each module was developed and verified against a standalone testbench, and at the top level, the wiring is just a long sequence of port maps. Since every stage is fully pipelined and accepts a new input every cycle, there is no need for handshaking or backpressure between stages.

The core detection chain starts with the camera infrastructure. `MIPI_BRIDGE_CAMERA_Config.v` brings up the MIPI bridge and the OV8865 sensor over I2C at boot, and `D8M_SET.v` converts the raw Bayer stream from the sensor into an 8-bit RGB stream at 640×480 along with the read-enable signal and the position counters `H_Cont` and `V_Cont`. From there, the stream passes through down-sampled grayscale conversion, Gaussian smoothing, Sobel edge extraction, sparse matched filtering, frame-level detection logic, and the VGA debug mux. The matcher output goes into `detection_logic.v`, which tracks the highest score across the frame and latches the detection result at the frame boundary. Finally, `vga_debug_mux.v` selects the user-visible debug view, writes matched-filter scores into a small M10K-based heatmap RAM, overlays the bounding box, and produces the final RGB stream sent out the VGA port. The internal matcher datapath is discussed in Section 5.4, where the score-tree approach is compared against the earlier systolic-array design.

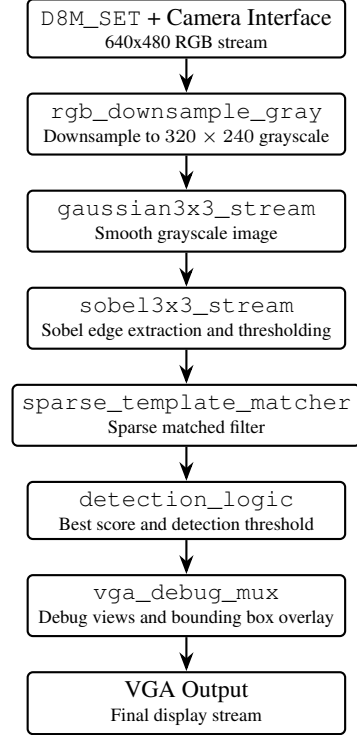


Figure 6: Main module integration for the final design.

Around this core pipeline sit several support modules that make the system usable on the board. `RESET_DELAY.v` synchronizes reset, the PLLs generate the camera and VGA clocks, the autofocus subsystem drives the camera lens over I2C, `FpsMonitor.v` reports the frame rate, and the seven-segment display logic shows the threshold and detection confidence. These modules do not change the pixel stream, but they support bring-up, debugging, and demos. Figure 6 shows the final top-level data path from camera capture to VGA output.

7. Testing and Evaluation

This section evaluates the performance of the completed object detection system and verifies that the design goals outlined earlier were achieved. Testing was conducted to validate the correct operation of the detection pipeline, measure real-time performance, and assess FPGA resource utilization. The results provided insight into the effectiveness of the design and highlighted the strengths and limitations of the final implementation.

7.1 Test Methodology

Testing was performed on the completed FPGA system using the DE1-SoC board and live video from the D8M camera module. The camera stream was displayed through VGA, and the debug display controls were used to switch between different views of the pipeline. This made it possible to ob-

serve the camera image, edge-detection output, matched-filter response, and final detection result directly on the monitor.

Before testing the full hardware system, individual modules were verified using Verilog testbenches. These testbenches applied known input values to each module and checked that the outputs matched the expected behavior. This helped confirm that the main parts of the design worked correctly before they were connected into the full video pipeline.

The module-level testbenches were used to check the major processing stages individually using separate testbenches. The front-end feature processing was tested to confirm that image features were generated correctly. The stream delay logic was tested to make sure pixel data was aligned correctly for the template window. The matched-filter scoring logic was tested to verify that the score calculation produced the expected result. The detection logic was tested to confirm that the highest score and detection location were selected correctly across a frame.

After the individual modules were tested, the full pipeline was evaluated on hardware. First, the camera and VGA output were checked to make sure live video was being captured and displayed correctly. The intermediate debug views were used to confirm that the processing stages were behaving as expected. The edge-detection view was used to check whether object outlines were visible, and the matched-filter heatmap was used to check whether high-score regions appeared near the target object.

Templates for the matched filter were generated from target object images using OpenCV. During testing, target objects were placed in front of the camera at approximately the same scale and orientation as the stored templates. The objects were also moved to different positions in the frame to check whether the detection logic could track the object location across the image.

Performance was evaluated using both on-board testing and Quartus reports. The VGA output was used to verify real-time behavior visually, while the frame-rate display was used to confirm that the system was operating near real-time video speed. Quartus timing reports were used to measure post-fit maximum clock frequency, and Quartus resource reports were used to measure ALM, DSP, and M10K usage. These tests were used to evaluate whether the design met the project goals of real-time operation, low-latency, and efficient FPGA resource usage.

7.2 Performance Metrics

The final design is evaluated using metrics that match the goals of a real-time FPGA video pipeline. The main goals are to keep the video output stable, meet the FPGA timing requirement, fit within the Cyclone V resources, and detect the target object consistently during live testing.

Frame rate. The system should maintain real-time video output at the VGA display rate. Since the pipeline processes one pixel per clock after the camera interface, the expected

output rate is tied to the 25 MHz VGA pixel clock and the 640×480 display timing.

Latency. Latency measures the delay between a pixel entering the processing pipeline and the corresponding detection/debug output being produced. This delay is mainly caused by the line buffers, Gaussian filter, Sobel filter, template-window delay, and pipelined score tree.

FPGA resource usage. Resource usage measures how much of the FPGA is required by the design. The main resources reported are ALMs, M10K memory blocks, DSP blocks, and registers. These values are taken from the Quartus Prime fit reports.

Timing performance. Timing performance is measured using the post-fit maximum clock frequency, or Fmax. The final design must meet the 25 MHz pixel-clock requirement with enough timing margin for stable VGA output.

Detection behavior. Detection behavior is evaluated using the live VGA debug views. The Sobel edge view shows whether the target outline is visible, the heatmap view shows where the matched-filter score is strongest, and the bounding-box overlay shows the final detected location.

False positives and robustness. False positives are checked by observing whether background edges trigger a detection when the target object is not present. Robustness is tested under normal lighting changes and cluttered backgrounds. Because the matcher operates on a binarized Sobel edge map, small changes in raw RGB intensity have less effect on the final match score.

7.3 Resource Utilization

Metric	Value
Detector Resolution	320×240
Template Size	64×64 / 64 sparse taps
Frame Rate	59.5 FPS
Latency	72 cycles / $2.88 \mu s$
ALMs	12%
DSP Blocks	23%
M10K RAM Blocks	29%
Post-fit Fmax	47.85 MHz
Power (TDP)	524.16 mW

8. Results

The final VGA debug views show the pipeline operating at each major stage. Figure 7 shows the live camera input before feature extraction. Figure 8 shows the Sobel edge view used by the sparse matched filter. Figure 9 shows the signed matched-filter score heatmap, where brighter regions indicate stronger template matches. Figure 10 shows the final detection output with the object location overlaid on the VGA display.

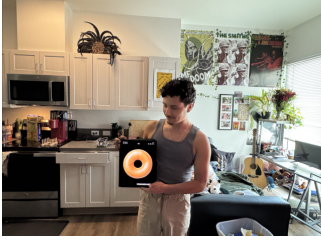


Figure 7: Original Camera Input RGB @ 640×480



Figure 8: Sobel Edge Output @ 320×240

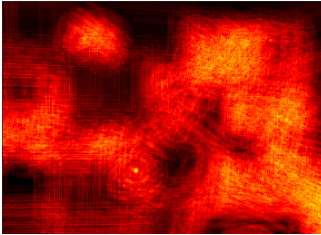


Figure 9: Matched Score Heatmap

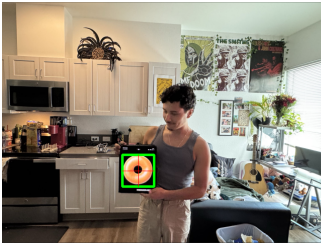


Figure 10: Detected Donut Output

9. Future Improvements

The current design works for fixed templates under stable lighting, but several extensions would make it more robust and more useful for future implementations:

- **Multi-scale matched filtering.** The biggest limitation right now is that the target has to be at roughly the same scale as the template. Running a small set of differently-sized templates in parallel, or downscaling the input before each filter, would let the board handle a wider

range of object sizes without exploding the hardware cost.

- **More efficient sparse templates.** The current 64×64 template uses 64 taps out of a possible 4096 positions. Improving the Python tap-selection script to choose the most informative positive and negative locations could either keep accuracy with fewer taps or improve accuracy with the same number.
- **CNN or learned-filter acceleration.** A small Convolutional Neural Network running on the same streaming pipeline would handle rotation and partial occlusion much better than a fixed template can. The multiply-accumulate hardware from the matched filter is structurally similar to what a small CNN needs, so most of the existing work could be reused rather than thrown away.
- **Improved thresholding.** Hysteresis thresholding on the matched-filter score, rather than a single threshold, would reduce fluttering in the bounding box when the score sits right at the cutoff. A confidence smoothing filter across frames would stabilize the detection further.
- **More optimized FPGA resource usage.** The current design uses about a third of the Cyclone V. Tighter resource packing would free up enough device resources for a second template to run in parallel, which would let the system track two object types at once without a separate bitstream using sequential convolution.

10. Conclusion

This project set out to demonstrate that real-time object detection can be implemented entirely in hardware on a modest FPGA, without the overhead of a general-purpose processor. The final design does this by streaming pixels from the D8M camera through a fully pipelined chain of RGB-to-grayscale conversion, Gaussian smoothing, Sobel edge detection, sparse matched filtering, and per-frame detection logic, with the result driving a VGA monitor in real-time. The system runs at 59.5 FPS with about 72 cycles of fixed processing latency and uses less than a third of the available device, while drawing a Quartus-reported 524 mW. The biggest takeaways from this work were about tradeoffs rather than algorithms. Matched filtering is a simple idea, but making it run on hardware requires choices at almost every step. We picked an L1 gradient approximation over an exact square root, a sparse template over a dense one, line buffers and tiling over full-frame storage, and a direct MAC tree over a general systolic array. Each of these traded some flexibility or accuracy for a real reduction in hardware cost, and each was necessary to fit within the Cyclone V budget while still meeting timing for live video. The streaming pipeline structure itself, where

every stage carries a valid signal and pixel coordinates alongside the data, was the single most important architectural decision because it made the modules independently testable and turned top-level integration into a port-mapping exercise rather than a debugging exercise. We also learned how different hardware design feels from software. There are no print statements to drop in, no quick patches once the bitstream is on the board, and timing closure is something you have to design for from the start rather than fix at the end. A bug in a module can change the layout enough to break Fmax on the next compile, so we learned to keep modules small, well-defined, and verified in simulation before integrating them. Quartus reports became a routine part of the workflow rather than an afterthought, and the resource budget shaped almost every algorithmic decision we made downstream. The system has clear limitations, most notably its dependence on the target being at the same scale and orientation as the template, but the path from this design to a more capable one is well-defined. The same pipeline structure, line buffers, and MAC-tree hardware can be reused for multi-scale detection or for a small learned model, which is the direction this design is set up to evolve toward. What we have at the end of the project is not just a working real-time detector but a platform that the next iteration can build on.

– Team Thomas

11. Contributions

Justin: Worked on the systolic engine, as well as designing the systolic version of the final matched filtering implementation. Ensured that the systolic array could perfectly fit with the existing project (so it could be instantiated in the top file instead of the score tree). Also developed the module for calculating the Sobel magnitude using the Sobel operator. Initially developed a Python script to use OpenCV for weight generation, but this script generated too many weights and exceeded the hardware capacity of the FPGA to fully make use of those weights while maintaining real-time processing.

Yaseen: Worked on the score tree for the final matched filtering implementation, including the logic used to combine sparse template tap results into a final detection score. Also helped develop the edge detection and thresholding portions of the pipeline, including the Sobel-based edge detector and the threshold comparison system used to produce a clean binary edge mask for object detection.

Isidro: Worked on the VGA debug multiplexer and detection logic for the final FPGA implementation. Helped write the pipeline outputs to the VGA display so the system could show the camera input, intermediate processing views, matched filtering response, and final detection result. Also worked on detection logic used to compare matched filter scores against the threshold and display the detected object location on the output screen.

Ricardo: Worked on camera capture and streaming video

pipeline, including the integration of the D8M camera module with the FPGA. Helped develop and verify the line-buffer architecture used throughout the convolution and assisted with hardware debugging and system integration to ensure real-time video processing. Also contributed to testing and validation of the edge-detection pipeline and helped evaluate design tradeoffs related to feature extraction and image processing.

Max: Helped with the final Sobel edge matched filter approach. This included shifting the design away from dense/raw image matching and toward a sparse template representation that only keeps the most useful object edge locations. Developed and tuned the template-generation workflow for real objects, such as the donut and cow, using hardware-aligned template outputs and sparse positive and negative selection. Helped make the design easier to configure by keeping template size, tap count, thresholds, and debug options parameterizable where possible. Developed simulation tests for the matched filter pipeline to verify score behavior, template alignment, and detection logic before synthesis.

Acknowledgments

The team would like to thank Professor Thomas for his guidance, feedback, and support throughout the senior design project. And our moms.

12. References

- [1] “Intel cyclone v fpgas.” <https://www.altera.com/products/fpga/cyclone/v>. Accessed: 2026-06-04.
- [2] Z. K. Baker, M. B. Gokhale, and J. L. Tripp, “Matched filter computation on fpga, cell and gpu,” in *15th Annual IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM 2007)*, (Napa, CA, USA), pp. 207–218, 2007.
- [3] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, J. Zambreno, and P. H. Jones, “Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels,” 2019.
- [4] S. Saini, B. Kasliwal, and S. Bhatia, “Comparative study of image edge detection algorithms,” *CoRR*, vol. abs/1311.4963, 2013.
- [5] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, “Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators.” <https://arxiv.org/abs/2110.03901>, 2021.
- [6] S. Centeno, C. Sprague, A. A. Purkayastha, R. Simar, and N. Magotra, “Exploring parallelism in fpga-based accelerators for machine learning applications,” 2025.
- [7] A. A. Purkayastha, J. Tharwani, and S. Aggarwal, “Fpga or gpu? analyzing comparative research for

- application-specific guidance,” 2025.
- [8] H. T. Kung, B. McDanel, and S. Q. Zhang, “Adaptive tiling: Applying fixed-size systolic arrays to sparse convolutional neural networks,” in *2018 24th International Conference on Pattern Recognition (ICPR)*, pp. 1006–1011, 2018.
 - [9] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
 - [10] M. Moghadam, S. Aygun, and M. H. Najafi, “Transc: Hardware-aware design of transcendental functions using stochastic logic,” 2026.
 - [11] Z. Fan, L. Zhang, and Y. Liu, “Nonlinear intensity, scale and rotation invariant matching for multimodal images,” 2023.
 - [12] R. Song, Z. Zhang, and H. Liu, “Edge connection based canny edge detection algorithm,” *Pattern Recognition and Image Analysis*, vol. 27, no. 4, pp. 740–747, 2017.
 - [13] Intel Quartus Prime, “Fitter Resource Usage Report for Canny Edge Detector Prototype,” 2026. Generated from project synthesis and fitting results.
 - [14] Intel Quartus Prime, “Fitter Resource Usage Report for Final Sobel Matched-Filter Design,” 2026. Generated from final DE1-SoC FPGA implementation.
 - [15] V. Nair, M. Chatterjee, N. Tavakoli, A. S. Namin, and C. Snoeyink, “Fast fourier transformation for optimizing convolutional neural networks in object recognition,” 2020.
 - [16] A. Dua, Y. Li, and F. Ren, “Systolic-cnn: An opencl-defined scalable run-time-flexible fpga accelerator architecture for accelerating convolutional neural network inference in cloud/edge computing,” 2020.
 - [17] V. Braeutigam, M. Stock, and B. Egger, “Template-based object detection using a foundation model,” 2026.
 - [18] Y. Yu, J. Xuan, S. Shi, X. Teng, S. Xu, and G. Tong, “Confident learning for object detection under model constraints,” 2026.